



Australian Government

Department of Defence

Defence Science and Technology Group

Programmatically generating a bidirectional assembler

SAPLING, November 2025

Kait Lam

DST Group / The University of Queensland



Goal: Assembly Mnemonic <-> Machine Code

`adds x1, x2, x3, lsl #4` <-> `0x8b031041`

- Goal: a framework for automatically generating instruction assemblers and disassemblers, given a machine-readable description of an architecture's assembly syntax.
- The generated assemblers should be invertible, hence “bidirectional”.
- Does not do *object* assembly (i.e., each instruction is (dis)assembled in isolation and it does not support symbolic addresses or labels).
- Starting with ARM.
- Unique benefits from MRA: completeness, formal backing.

Background: LLVM, Capstone, and Keystone

- llvm-mc command-line tool, but difficult to embed or distribute.

```
echo 'catal w0, w4, [x3]' | llvm-mc --arch=arm64 --mattr=v9.5a --show-encoding
```

```
echo '0x0C 0x00 0x80 0xF9' | llvm-mc -arch=arm64 -mattr=v9.5a --disassemble
```

- Existing projects try to extract LLVM's logic into reusable components:
 - Capstone disassembly engine
 - Keystone assembly engine
- However, still a very complicated and manual process.
- In the case of Keystone, has no commits in the last 2 years and no releases in the past 5 years.
- Missing assembly instructions which we are interested in.

Background: ARM Architecture Reference Manual

Contents

- Back to search
- All Armv8-A Documentation
- Arm A64 Instruction Set Architecture
- Base Instructions
 - ADC: Add with Carry.
 - ADCS: Add with Carry, setting flags.
 - ADD (extended register): Add (extended register).
 - ADD (immediate): Add (immediate).
 - ADD (shifted register): Add (shifted register).**
 - ADDG: Add with Tag.
 - ADDS (extended register): Add (extended register), setting flags.
 - ADDS (immediate): Add (immediate), setting flags.
 - ADDS (shifted register): Add (shifted register), setting flags.
 - ADR: Form PC-relative address.
 - ADRP: Form PC-relative address to 4KB page.
 - AND (immediate): Bitwise AND (immediate).
 - AND (shifted register): Bitwise AND (shifted register).
 - ANDS (immediate): Bitwise AND (immediate), setting flags.
 - ANDS (shifted register): Bitwise AND (shifted register), setting flags.
 - ASR (immediate): Arithmetic Shift Right (immediate); alias of ASRV.
 - ASRV: Arithmetic Shift Right Variable.
 - AT: Address Translate: an alias of SYS.
 - AUTDA, AUTDA1716: Authenticate Data address, using key A.
 - AUTDB, AUTDB1716: Authenticate Data address, using key B.
 - AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIAZ1716: Authenticate...
 - AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIBZ1716: Authenticate...
 - AXFLAG: Convert floating-point condition flags from Arm to...
 - B: Branch.
 - B.cond: Branch conditionally.
 - BFC: Bitfield Clear: an alias of BFM.
 - RFI: Bitfield Insert: an alias of...

ADD (shifted register)

ADD (shifted register) adds a register value and an optionally-shifted register value, and writes the result to the destination register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	
sf	0	0	0	1	0	1	1	shift	0	Rm						imm6											
op = 5																											

32-bit (sf == 0)

ADD <Wd>, <Xn>, <Xm>{, <shift> #<amount>}

64-bit (sf == 1)

ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}

```
integer d = UInt(Rd);
integer n = UInt(Rn);
integer m = UInt(Rm);
integer datasize = if sf == '1' then 64 else 32;
boolean sub_op = (op == '1');
boolean setflags = (S == '1');

if shift == '11' then UNDEFINED;
if sf == '0' & imm6<5> == '1' then UNDEFINED;

shift = shift_type = DecodeShift(shift);
integer shift_amount = UInt(imm6);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in "shift":
shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

1. Bitfield pattern

3. Asm template

2. Encoding symbols

 We use the XML machine-readable architecture (MRA), which is used to generate the web pages.

Background: ARM Architecture Reference Manual

Contents

< Back to search

< All Armv8-A Documentation

Arm A64 Instruction Set Architecture

Base Instructions

ADC: Add with Carry.

ADCS: Add with Carry, setting flags.

ADD (extended register): Add (extended register).

ADD (immediate): Add (immediate).

ADD (shifted register): Add (shifted register).

ADDG: Add with Tag.

ADDS (extended register): Add (extended register), setting flags.

ADDS (immediate): Add (immediate), setting flags.

ADDS (shifted register): Add (shifted register), setting flags.

ADR: Form PC-relative address.

ADRP: Form PC-relative address to 4KB page.

AND (immediate): Bitwise AND (immediate).

AND (shifted register): Bitwise AND (shifted register).

ANDS (immediate): Bitwise AND (immediate), setting flags.

ANDS (shifted register): Bitwise AND (shifted register), setting...

ASR (immediate): Arithmetic Shift Right (immediate): an alias...

ASR (register): Arithmetic Shift Right (register): an alias of ASRV.

ASRV: Arithmetic Shift Right Variable.

AT: Address Translate: an alias of SYS.

AUTDA, AUTDZA: Authenticate Data address, using key A.

AUTDB, AUTDZB: Authenticate Data address, using key B.

AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA: Authenticate...

AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIBZ: Authenticate...

AXFLAG: Convert floating-point condition flags from Arm to...

B: Branch.

B.cond: Branch conditionally.

BFC: Bitfield Clear: an alias of BFM.

RFI: Bitfield Insert: an alias of

ADD (shifted register)

Add (shifted register) adds a register value and an optionally-shifted register value, and writes the result to the destination register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5
sf	0	0	0	1	0	1	1	shift	0	Rm						imm6										
op	S																									

32-bit (sf == 0)

ADD <Wd>, <Wn>, <Wm>{, <shift> #<amount>}

64-bit (sf == 1)

ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}

```
integer d = UInt(Rd);
integer n = UInt(Rn);
integer m = UInt(Rm);
integer datasize = if sf == '1' then 64 else 32;
boolean sub_op = (op == '1');
boolean setflags = (S == '1');

if shift == '11' then UNDEFINED;
if sf == '0' && imm6<S> == '1' then UNDEFINED;

ShiftType shift_type = DecodeShift(shift);
integer shift_amount = UInt(imm6);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<shift>	Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in "shift":
shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

(1) Bitfield pattern

Add (shifted register) adds a register value and an optionally-shifted register value, and writes the result to the destination register.

31	30	29	28	27	26	25	24	23	22	21	20	19
sf	0	0	0	1	0	1	1	shift		0	Rm	
op		S										

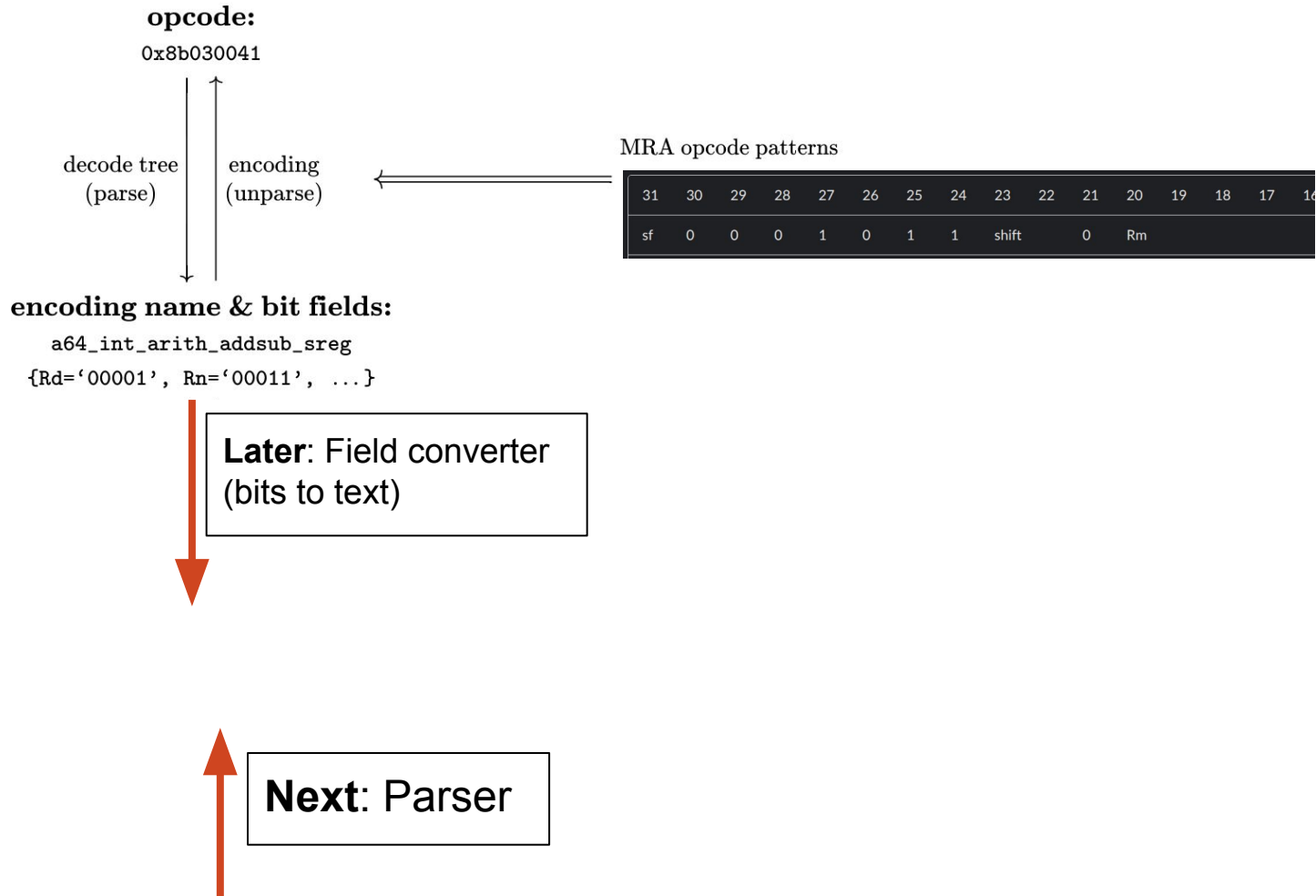
(2) Assembler encoding symbols

Assembler Symbols	
<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

(3) Asm template pattern

```
ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Approach



Goal: `add x1, x2, x3 -> { Xd="x1"; Xm="x3"; Xn="x2" }`

Invertible Parsing

- Parses text into a **bindings map** — mapping field names to strings.
- Used for:
 - parsing opcode into bit fields, and
 - parsing assembly into template fields (and vice-versa).
- Restricted grammar to allow for “unparsing” (printing).
- Only has sequential composition, (disjoint) union, and terminals.

```
type parseable =  
  (** A choice between a given list of alternatives. *)  
| Or of parseable list  
  (** A sequential composition of the given parseables. *)  
| Seq of parseable list  
  
  (** A subparser, binding its result into the associated name. See {!type:bind}. *)  
| Bind of bind  
  
| Lit of string (** A literal string. *)  
| Space (** Zero or more within-line space characters (space or tab). *)  
| Digits (** One or more decimal digits (without sign characters). *)  
| Eof (** A parser that succeeds only at the end of the text. *)
```


Invertible Parsing — Example

ADD_64_addsub_shift

"ADD" <Xd> ", " <Xn> ", " <Xm> (", " <shift> "#" <amount>)?

Example 1:

add x1, x2, x3, LSL #4

```
bindings = {  
  <Xd> = [["X", "1"]];  
  <Xm> = [["X", "3"]];  
  <Xn> = [["X", "2"]];  
  <amount> = [["4"]];  
  <shift> = [["LSL"]];  
  ADD_64_addsub_shift = [[]];  
}
```

Example 2:

add x1, x2, x3

```
bindings = {  
  <Xd> = [["X", "1"]];  
  <Xm> = [["X", "3"]];  
  <Xn> = [["X", "2"]];  
  ADD_64_addsub_shift = [[]];  
}
```

- Presence / absence of bindings is enough to uniquely determine a path through the “or” choices of the parsers.
 - Assumption we make of the MRA.
- We *just* have to produce the right text for each binding, then we can unparse to get the assembly text.

Approach



Goal: { Rd=0b0001; ... } -> { Wd="w1"; ... }

Bidirectional Field Converters

Asm template field

Bitfield

<Wd>

Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

- Lots of implicit things, e.g.:
 - “32-bit” “general-purpose register” means register beginning with “w” and numbered 0–30,
 - “encoded” means an integer-binary conversion,
 - no mention of stack pointer means “wzr” is the all-ones encoding (in place of number 31).

Bidirectional Field Converters — Syntax

Bidirectional syntax	Forwards (→)	Backwards (←)
<pre>Rd <--> BitsToUint(5) <--> N; ^ ^ ^ var intrinsic var</pre>	<pre>read(Rd) > BitsToUint(5) > write(N)</pre>	<pre>write(Rd) < BitsToUint^{ -1 }(5) < read(N)</pre>
<pre>N <--> 31;</pre>	<pre>assert N == 31;</pre>	<pre>N := 31;</pre>
<pre>A; B;</pre>	<pre>A; B;</pre>	<pre>B; A;</pre>
<pre>choice { A; } or { B; }</pre>	• Non-deterministic choice between two or more alternatives. • Exactly one alternative should succeed, guarded by assertions or partial intrinsics.	
<pre>decl v1, ..., vn;</pre>	• Ensures that the given variables are defined. • At beginning, this acts to declare the <i>input</i> variables. • At end, this acts to declare <i>output</i> variables.	

Goal: {Rd=0b0001; ...} -> {Wd="w1"; ...}

Bidirectional Field Converters — Example

<Wd>

Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

```
(* Declares the input variable to be Rd. This is the input in the forwards direction
   and the output in the backwards direction. *)
decl Rd; (* bitstring *)

Rd <-> BitsToUInt(5) <-> N;

choice {
  (* This branch will continue only when N is 31. *)
  N <-> 31;
  "wzr" <-> Wd;
} or {
  (* NotIn asserts that N is not equal to 31. *)
  N <-> NotIn([31]) <-> N;

  (* Integer to decimal string. *)
  N <-> IntToDecimal <-> n;

  (* Concatenates the string arguments. Type parameters are the width of each
     string component, with None meaning the remainder of the string. *)
  ("w", n) <-> Concat(1, None) <-> Wd;
}

(* Declares the output of the forwards direction as Wd. *)
decl Wd; (* text string *)
```

How do we go from

<Wd>

Is the

al-pu

regex

to

```
(* This branch will co  
N <-> 31;  
"wzr" <-> Wd;  
} or {  
(* NotIn as  
N <-> N  
to decimal string. *)  
N <-> IntToDecimal <-> n;  
(* Concatenates the string arguments. Typ  
string component, with None meaning t  
("w", n) <-> Concat(1, None) <-> Wd;  
}  
(* Declares the output of the forwards direction as Wd. *)  
decl Wd;
```

Related work

- Alastair Reid's work on *Bidirectional ARM Assembly Syntax Specifications*.
- Ghidra's Sleigh (itself a continuation of SLED)
- LLVM TableGen
- Bidirectional transformations: lenses, etc.
- What is ARM going to do with ASLv1? Will it improve this situation?
- Sail has a language feature for bidirectional mapping of instruction classes.

Future work

- Not covered in this presentation:
 - How do you choose between all possible instructions? — Big “or” of all options.
 - Abstract interpretation of Bidir language to get parsers for fields.
- Future work:
 - Omitting defaults in emitted assembly syntax.
 - Handling “aliases” in assembly syntax.
 - ^-- Both seem hard!
 - “Long tail”
- Hard things:
 - “<amount>: is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the “imm3” field. It **must be absent when** <extend> is absent, **is required when** <extend> is LSL, and **is optional when** <extend> is present but not LSL.”

CMN <Xn>, <Xm>{, <shift> #<amount>}

is equivalent to

ADDs XZR, <Xn>, <Xm>{, <shift> #<amount>}

and is always the preferred disassembly.

Thanks!



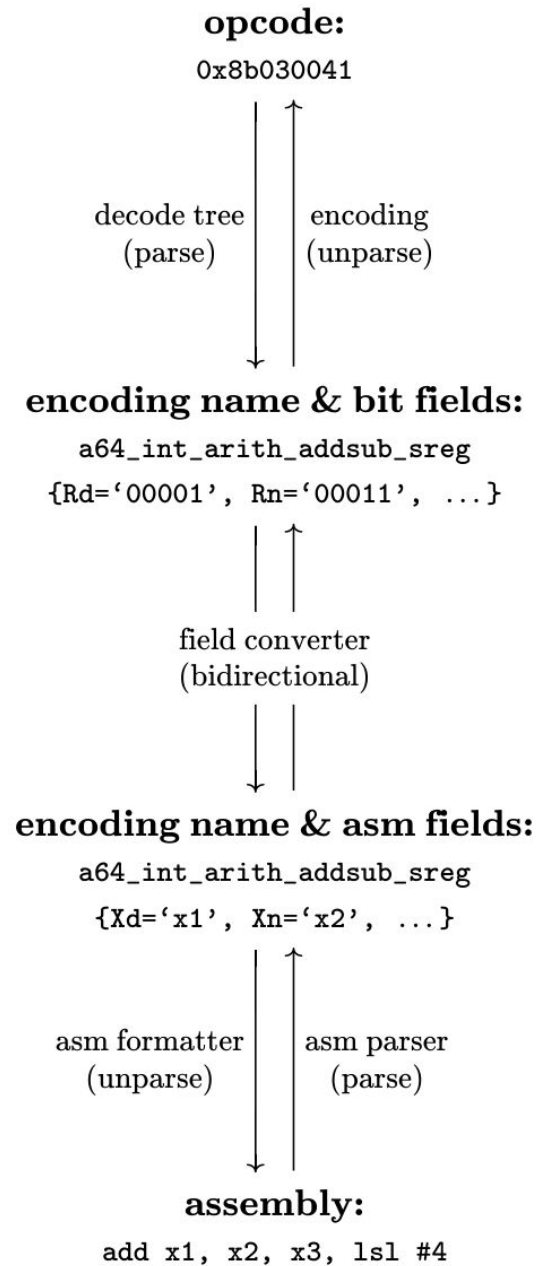
github.com/katrinafyi/mra-bi-assembler

... also see the docs website:

katrinafyi.github.io/mra-bi-assembler/mra_bi_assembler/

... our other, more well-known, work using ARM MRA:

github.com/UQ-PAC/aslp



Stop

Hard: BFM aliases

<https://developer.arm.com/documentation/ddi0596/2021-03/Base-Instructions/BFI--Bitfield-Insert--an-alias-of-BFM-?lang=en>

64-bit ($\text{sf} == 1 \ \&\& \ N == 1$)

$\text{BFI } \langle \text{Xd} \rangle, \langle \text{Xn} \rangle, \# \langle \text{lsb} \rangle, \# \langle \text{width} \rangle$

is equivalent to

$\text{BFM } \langle \text{Xd} \rangle, \langle \text{Xn} \rangle, \#(-\langle \text{lsb} \rangle \text{ MOD } 64), \#(\langle \text{width} \rangle - 1)$

and is the preferred disassembly when $\text{UInt}(\text{imms}) < \text{UInt}(\text{immr})$.

Alias	Is preferred when
BFC	$\text{Rn} == '11111' \ \&\& \ \text{UInt}(\text{imms}) < \text{UInt}(\text{immr})$
BFI	$\text{Rn} != '11111' \ \&\& \ \text{UInt}(\text{imms}) < \text{UInt}(\text{immr})$
BFXIL	$\text{UInt}(\text{imms}) \geq \text{UInt}(\text{immr})$

Hard: conditional absent/required/optional/preferred

<https://developer.arm.com/documentation/ddi0596/2021-03/Base-Instructions/ADD--extended-register---Add--extended-register--?lang=en>

For the 64-bit variant: is the extension to be applied to the second source operand, encoded in "option":

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL UXTX
100	SXTB
101	SXTH
110	SXTW
111	SCTX

If "Rd" or "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount>

Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

Hard: no bindings inside optional, adjacent registers

<https://developer.arm.com/documentation/ddi0596/2021-03/Base-Instructions/CASP--CASPA--CASPAL--CASPL--Compare-and-Swap-Pair-of-words-or-doublewords-in-memory-?lang=en>

64-bit CASPL (sz == 1 && L == 0 && o0 == 1)

CASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]

Background: Capstone/Keystone

- Capstone: Disassembly engine
- Keystone: Assembly engine
- Tries to extract LLVM's assembly parsing/formatting logic into reusable components.
- However, still a very complicated build system.
- In the case of Keystone, has no commits in the last 2 years and no releases in the past 5 years.
- Missing instructions which we are interested in.

Background: ARM Architecture Reference Manual

Consider `add x1, x2, x3, LSL 4` = 41 10 03 8B.

0x 8 b 0 3 1 0 4 1
1000 1010 0000 0011 0001 0000 0100 0001
xxx0 1011 xx0x xxxx xxxx xxxx xxxx
sf | S shift Rm imm6 Rn Rd
op

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sf	0	0	0	1	0	1	1	shift	0	Rm					

Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in "shift":	
shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

Background: ARM ARM — Encodings

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in the "shifter" field.

<shift>	shift	<shift>
	00	LSL
	01	LSR
	10	ASR
	11	RESERVED

<amount> For the 32-bit variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.

For the 64-bit variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.